Finetuning

Language Models

Course Contents

Deep Learning
Transformer
LLMs
Context Engineering
Finetuning
Agents
The Bigger Picture

This Part: Finetuning

- transfer learning
- text classification
- instruction tuning
- parameter-efficient finetuning
- preference alignment

Modern Language Models in a Nutshell

tokenization: split text into chunks

pretraining: self-supervised, next- or masked-token prediction

- vector embeddings from tokens (semantic meaning)
- add positional encoding (attention permutation-invariant)
- attention mechanism (transformer) → contextual embeddings

finetuning: specialization

classification model (specific task):
finetuning on labeled data set (often using encoder-only transformer)

assistant/chatbot (multipurpose):
instruction tuning (decoder-only transformer), potentially followed by alignment

Transfer Learning from Foundation Models

idea:

- pretrain a big model (called foundation model) on a broad data set
- then use these learnings for subsequent trainings on specific (typically narrow) data by means of finetuning or feature extraction

→ here: self-supervised, internet-scale pretraining of models like BERT or GPT

works well for homogenous, unstructured data (e.g., text, images)
but very difficult for heterogenous, structured/tabular data

Finetuning of BERT

- get pretrained BERT as foundation model (MLM pretraining: loss computed against projection from last hidden state of [MASK] token to vector of logits)

- remove pretraining classification head (→ get contextual token embeddings)
- add another, task-specific classification head (see a few examples on the following slides)

- train (finetune) with data for actual task at hand (starting with random weights for new classification head and pretrained weights for the rest)
- options: finetune all pretrained weights or only a part (freezing the others)

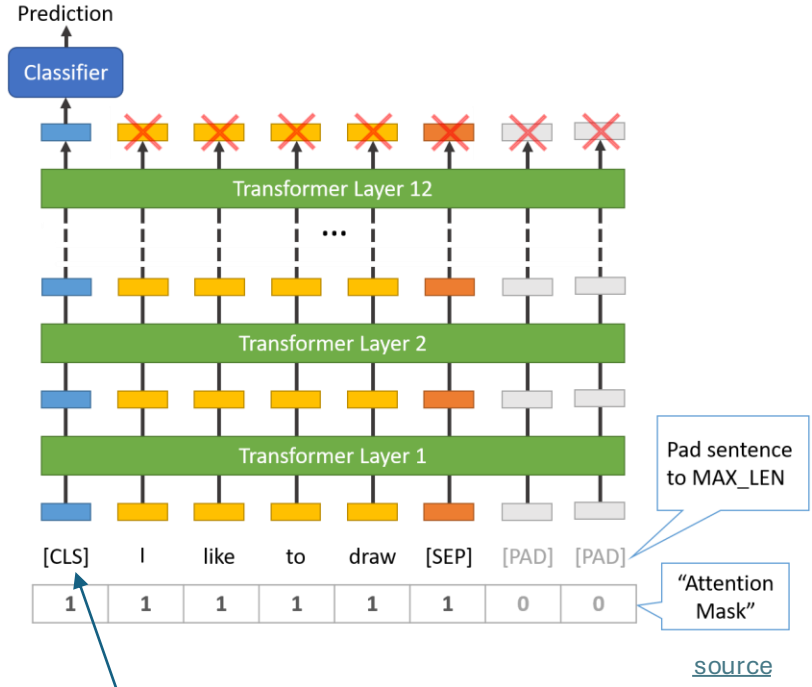
Example: Sequence Classification

e.g., classify sentiment or topic

- a special [CLS] token is prepended to the input (already present in pretraining)
- its final hidden state (vector embedding) is passed into a classification head (resulting in logits over different categories)

→ [CLS] embedding is finetuned to summarize the whole sequence

(alternative without [CLS]: average pooling)



Next-Sentence Prediction (NSP) in BERT

BERT uses NSP as pretraining objective

(in addition to MLM → total loss is sum of MLM and NSP losses)

goal: help BERT learn relationships across sentences (later dropped)

- training examples: [CLS] Sentence A [SEP] Sentence B [SEP]
- [CLS]: special token to represent the whole sequence
- hidden representation of [CLS] fed into classification head predicting whether B is the actual next sentence or not

BERT Variants

RoBERTa: improved performance

- removed next sentence prediction as pretraining objective
- ~ten times more training data, longer training, larger batches, longer sequences
- dynamic masking: instead of fixed masked tokens for each sequence, masks change each epoch

DistilBERT: much lighter and faster, retaining ~95% of BERT's performance

- a smaller "student" model is trained to mimic BERT's predictions
- 40% fewer parameters than BERT, 60% faster inference, using half the memory

Example: Sentence Embeddings

use case: sentence similarity / retrieval (see RAG)

BERT optimized for NSP, not for mapping independent sentences into a comparable vector space → need to finetune on considered similarity label

- paraphrase (yes or no)
- natural language inference (entailment/contradiction/neutral)
- semantic textual similarity (human-judged similarity score) → regression loss

approach with vanilla BERT:

- input: [CLS] Sentence A [SEP] Sentence B [SEP]
- [CLS] token embedding as aggregate representation of the pair
- classifier/regressor head trained directly on considered similarity label

SBERT

issue with vanilla BERT:

prediction of similarity score requires each pair of sentences to be processed together (no independent sentence embeddings) → $O(n^2)$ for n sentences

SBERT: instead of finetuning vanilla BERT,

- uses two (or three) copies of same BERT model (with shared weights)
- passes each sentence through one of the copies to produce its embedding from last hidden states (use [CLS] token or average pooling)
- puts common head on top of copies, optimizing $\text{similarity}(u, v) \approx \text{label}(u, v)$ of sentence embeddings u and v (→ different SBERT versions for different similarity tasks/labels)

→ cosine similarity (or dot product) can then be used to compare independent sentence embeddings

Example: Token Classification

examples of use cases:

- part-of-speech tagging: assigning each word in a sentence its grammatical role (or part of speech) such as noun, verb, etc.
- named entity recognition: identifying and classifying named entities (proper nouns) in text into predefined categories like Person, Organization, Location, Date, etc.

per-token predictions needed → no pooling or [CLS] usage

instead, just project each token's hidden state independently into logits and sum the per-token losses in each sequence

Finetuning Scenarios

features more generic in early layers (e.g., token structure, syntax, part-of-speech) and more specific to the pretraining data in later layers (e.g., word sense, entity linking)

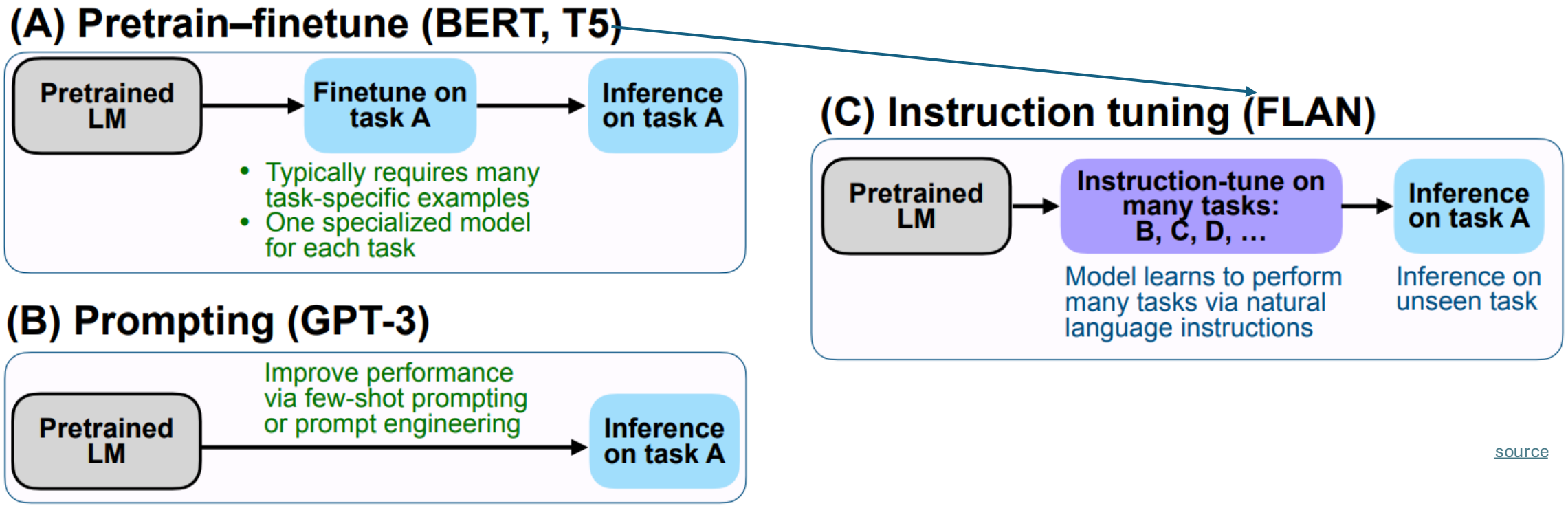
→for small finetuning data sets, typically keep weights of early layers fixed to avoid overfitting

→for large finetuning data sets (less danger of overfitting), finetuning all layers can be beneficial

Instruction Tuning

instruction tuning is a form of supervised finetuning (SFT)

so is the finetuning discussed before (although usually just called finetuning)



source

Instruction Tuning Data

instruction tuning = SFT with instruction–response data

Instruction: "Summarize this article in one sentence."

Output: "The article discusses the effects of climate change on polar bears."

instruction tuning data fed to model as input sequences, just like in pretraining

objective still next-token prediction (language modeling)

structure of text sequences only thing telling the model where instruction ends and response begins

also learns formatting: desired patterns embedded in instruction–response pairs

Base vs Instruction-tuned LLM

Explain why the sky is blue, as if you were talking to a curious 10-year-old.

base LLM response:

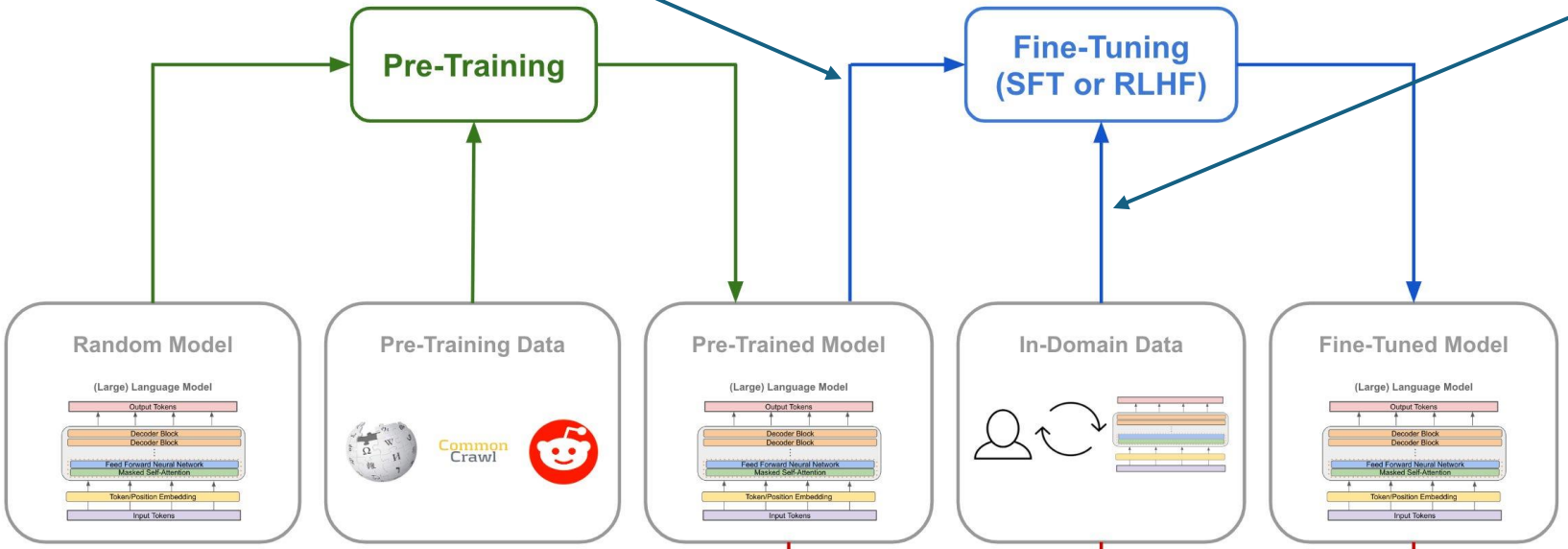
“The sky is blue because of the way light interacts with the atmosphere. Sunlight contains many colors, and when it passes through air molecules, the blue light scatters more than other colors. That’s why we see the sky as blue during the day.”

instruction-tuned LLM response:

“Imagine the sky is like a giant ocean of tiny invisible balls called air molecules. When sunlight shines through, the blue light bounces around more than the other colors, kind of like how tiny balls make waves in a pool. That’s why, when you look up, the sky looks blue! And at sunset, the light has to travel through more air, so the reds and oranges shine through instead.”

generic “instruct“ versions of LLMs
(potentially also aligned)
e.g., Llama-2-7b-chat, Mistral-7B-Instruct

expert LLMs



base LLM: poor instruction following,
no proper multi-turn converstations
(e.g., Llama-2-7b, Mistral-7B)

RAG

ready to become
an LLM chatbot

Use Cases of Instruction Tuning

- task specialization (e.g., code generation)
- domain-specific instructions (e.g., interpret molecular formulas)
- style or tone adaptation (e.g., corporate writing style, friendly chatbot tone)
- reasoning (CoT inputs) → reasoning models

Parameter-Efficient Finetuning (PEFT)

LLMs have billions of parameters

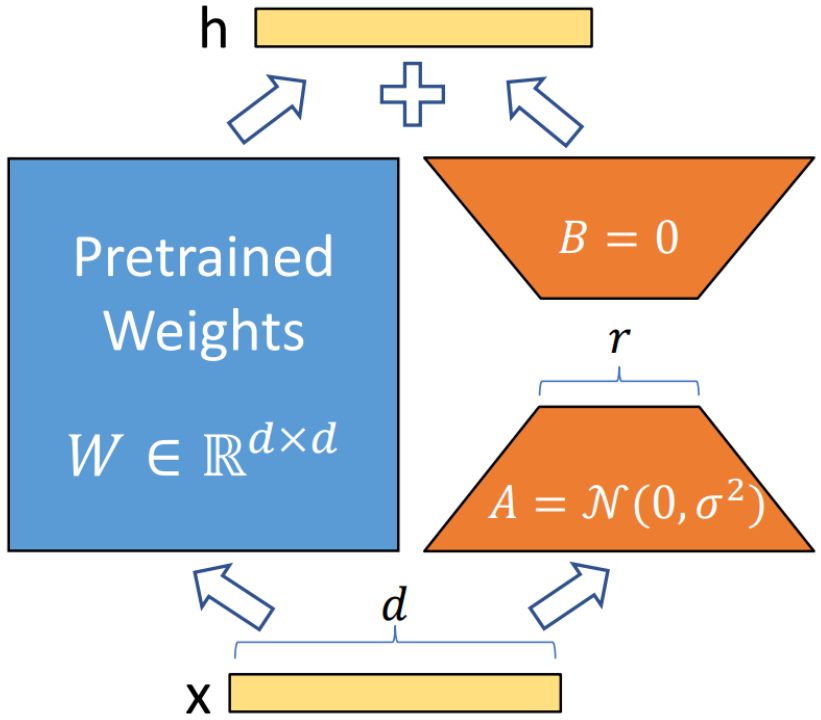
→ full finetuning is very expensive

PEFT: adapt LLMs to new tasks without updating all parameters, instead learning small, efficient modifications

popular method: Low-Rank Adaptation (LoRA)

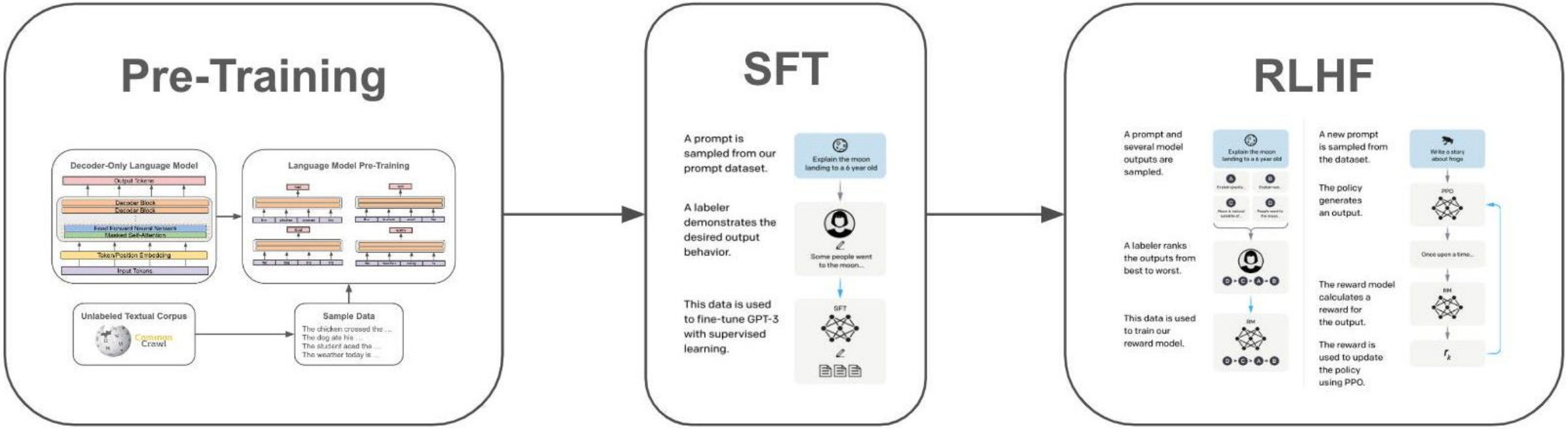
often combined with quantization (representing model weights and/or activations with lower-precision numbers) to further reduce memory usage

LoRA:



Preference Alignment

RLHF can achieve both instruction following and preference alignment



here: GPT-3

source

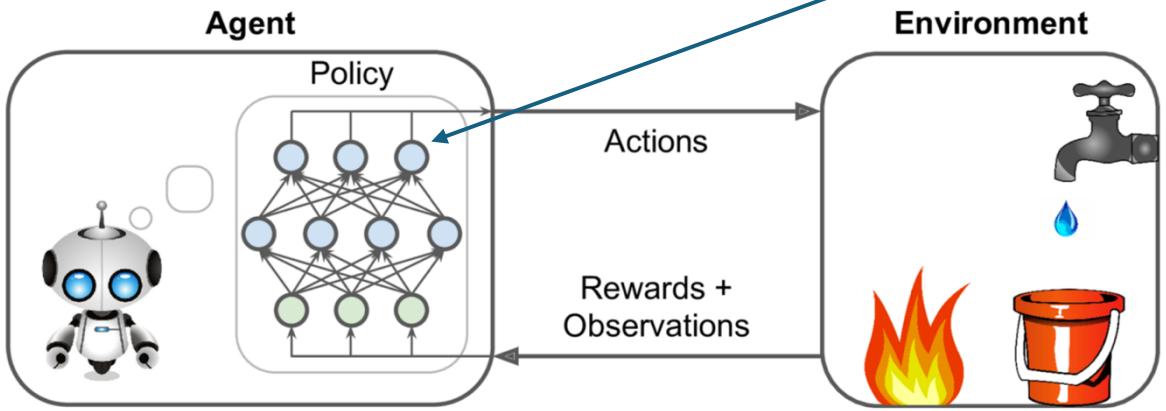
idea: aligning LLM output with human preferences

InstructGPT: SFT followed by reinforcement learning from human feedback (RLHF)

Aside: Policy Gradient Method

rationale for using reinforcement learning: allows to train on whole responses with global feedback, something next-token prediction cannot do naturally

in general: policy network with actions as output (here: next-token probabilities)



loss: $-\log P(a_t|s_t;\boldsymbol{w}) \cdot R$

return (here: just one reward for entire sampled text sequence)

gradient ascent

model for action given state (here: LLM)

GPT-3 → InstructGPT → ChatGPT

dialog system (aka conversational agent or chatbot): any system designed to communicate with humans using natural language

with instruction-tuned and aligned LLMs: assistant-style models

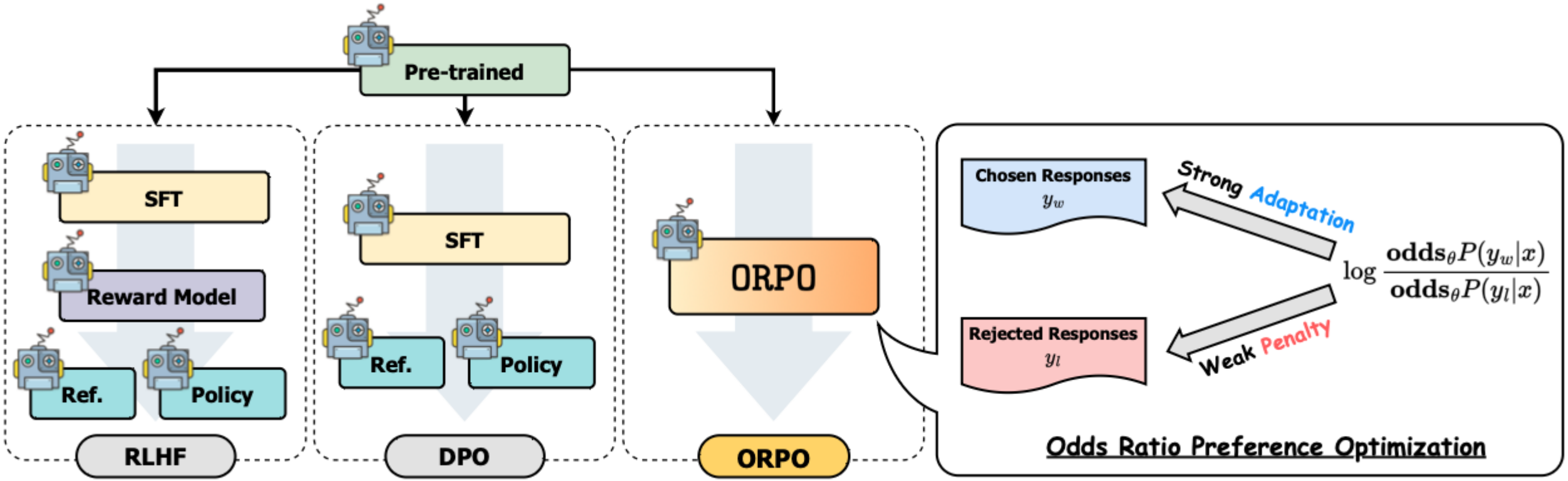
crucial: multi-turn dialogue

→ passing conversation history as part of input to LLM at each step

lightweight PyTorch re-implementation of ChatGPT: nanochat

Other Methods for Preference Alignment

alternatives to RLHF: DPO, ORPO



Group Relative Policy Optimization (GRPO)

goal: efficient RL approach

- developed by DeepSeek to scale RL for Reasoning Models (e.g., DeepSeek-R1)
- optimizes the RLHF process instead of removing RL (unlike DPO/ORPO)

core problem with standard PPO:

- requires a separate Critic Model to estimate the reward baseline
- the Critic is as large as the Actor (Policy) model → huge memory bottleneck

GRPO solution: drop the critic

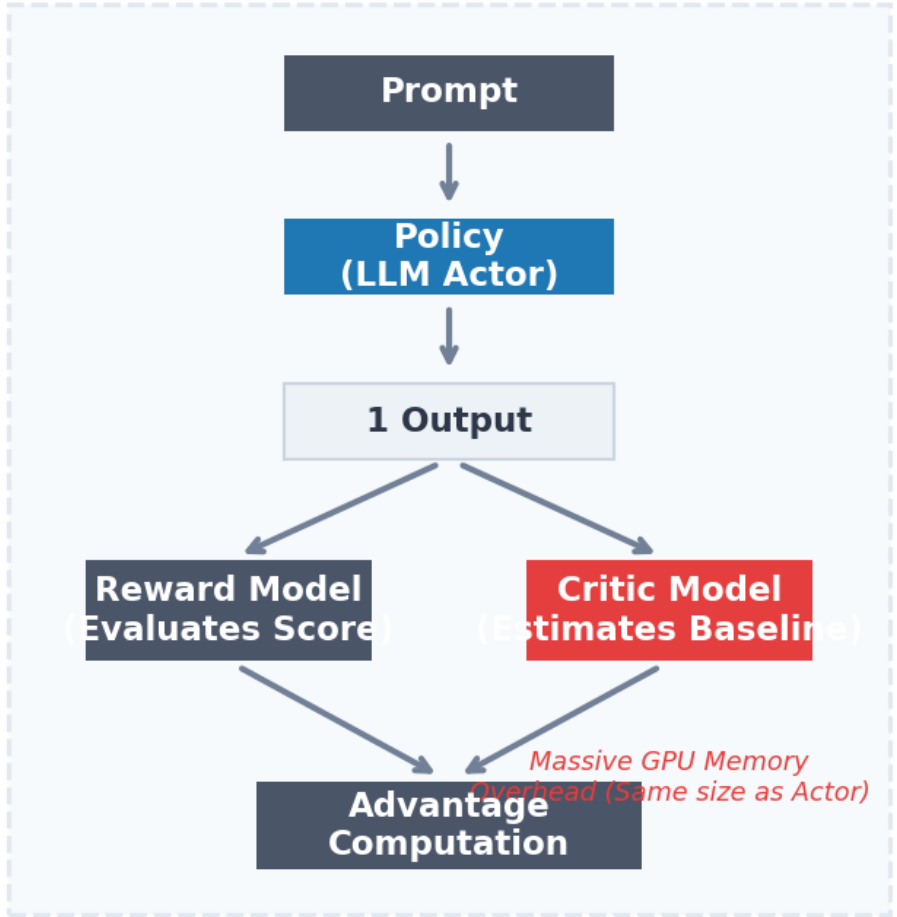
- group generation: policy generates a group of N outputs for a single prompt
- relative reward: rewards are scored for all outputs and normalized within the group
- mathematical baseline: the group average replaces the critic model entirely

key benefits:

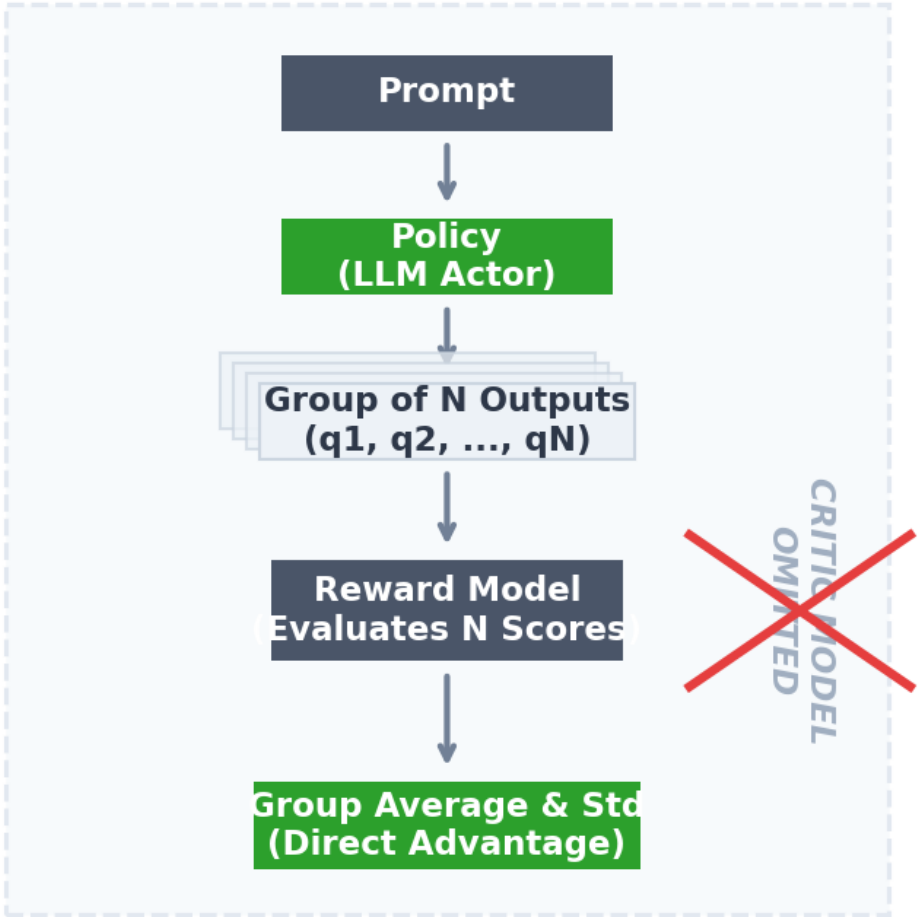
- ~50% GPU memory savings: allocates more VRAM to sequence length (critical for long chain-of-thought reasoning)
- stable training: compares answers against each other rather than an absolute scale

Architectural Difference: PPO vs. GRPO

Standard PPO (RLHF)

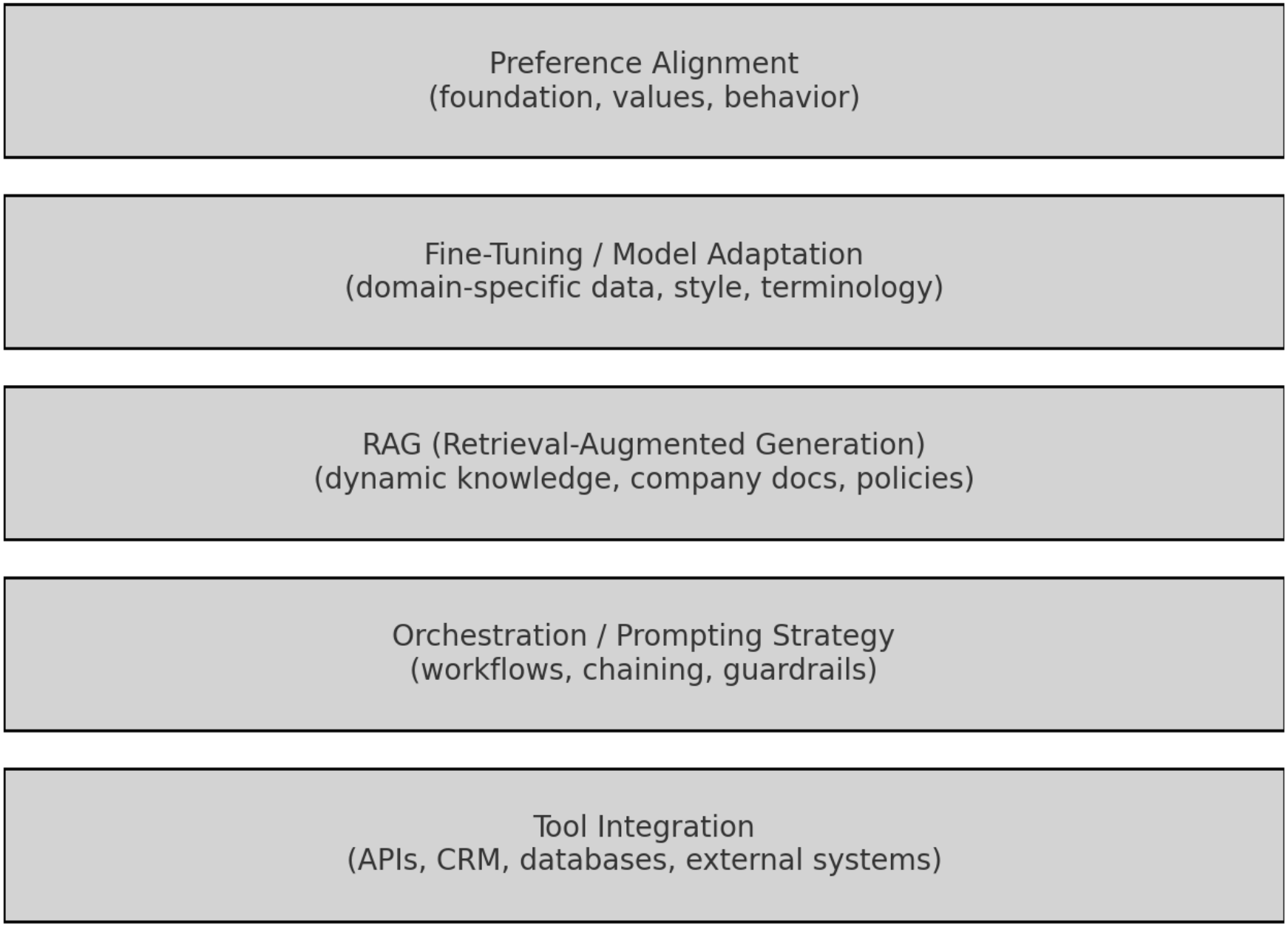


GRPO (Group Relative)



Recovers ~50% GPU memory compared to PPO by calculating baseline statistically

Layers of Adapting a Chatbot to Company Needs

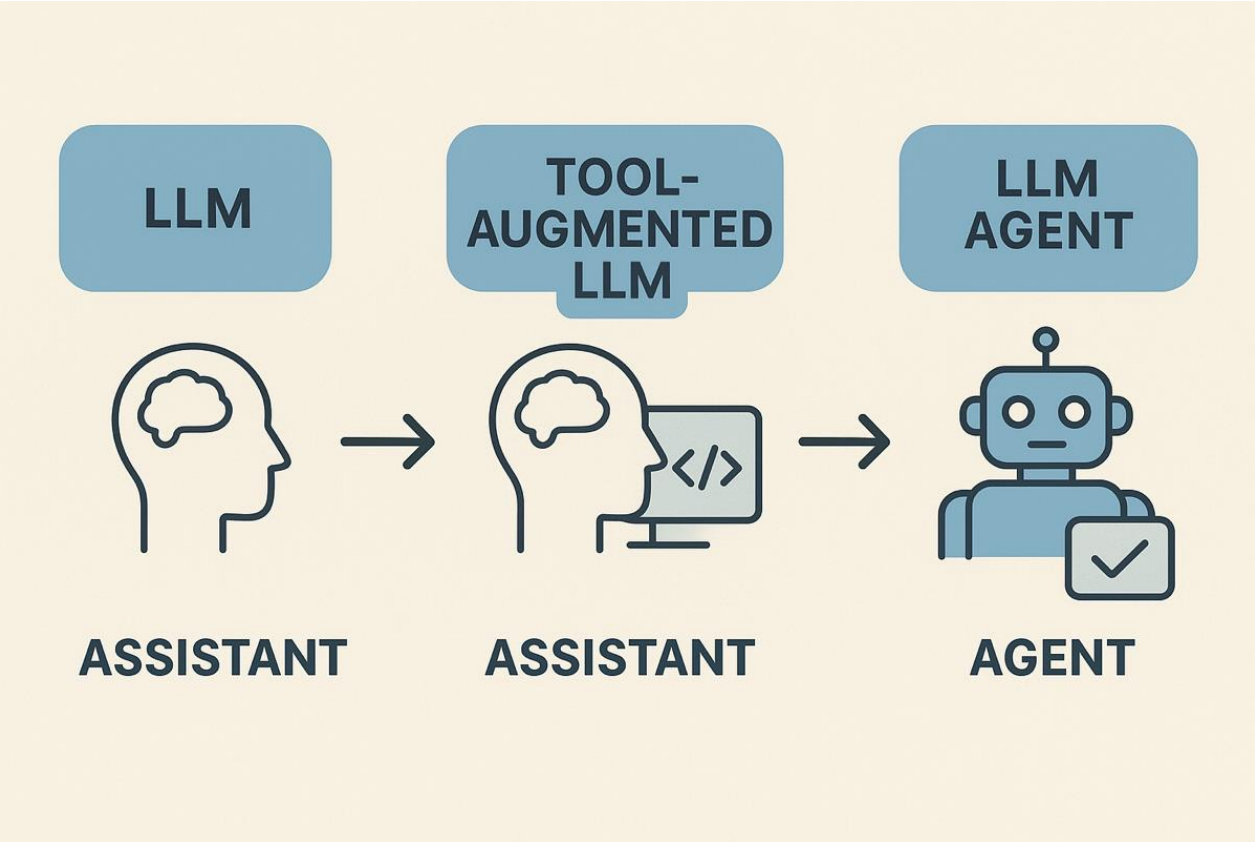


Next: Agents

latest AI evolutionary step: **LLMs as assistants** to **LLMs as agents**

passive, reactive

responding to user prompts



active, autonomous, decision-making entities

acting on goals or tasks